

Software Requirements Document

Solari — AI-Powered Solar Industry Chatbot

Track 2 — Project Group AI_02

Prepared by: Ayesha Naveed

Document type: Requirements Specification

Date: August 2026

A quick note before you dive in: this document is written from our own team's perspective — it reflects what we actually built and why, not a generic template.

1. Introduction

1.1 Purpose

This document lays out what Solari — our AI-powered solar consultation assistant — actually needs to do. We put it together so that everyone on the team, along with anyone evaluating the project, is working from the same understanding of the system's behavior before we finalize things for submission. It's meant to be read before implementation review, not after.

1.2 Project Overview

Solari is a web-based chatbot we built to help residential, commercial, and industrial customers in Pakistan make sense of solar PV systems — get a rough sizing estimate, ask about panels and inverters, and get answers that are actually grounded in real research rather than guesswork. Because the assistant deals with billing and savings claims, we made sure it stays strictly within the 2026 NEPRA Net-Billing framework and enforces a few non-negotiable safety guardrails around electrical hazards, panel cleaning, and the kind of “zero bill” promises that oversell what solar can do.

Under the hood, it's a static HTML/CSS/JavaScript frontend talking to a Python FastAPI backend. Every AI reply is grounded in three local knowledge-base documents we compiled (hardware, systems, and operations research), and whenever someone asks for a system size, the AI doesn't do the math itself — it calls a dedicated, deterministic sizing function so the numbers stay consistent and reliable.

1.3 Intended Audience

- Our own development team — for implementing and testing each module.
- Project supervisors and evaluators reviewing our Track 2 submission.
- Anyone who picks this project up later and needs to understand how it's supposed to behave before changing it.

1.4 Scope

What Solari actually covers: a conversational chat interface, an AI consultation engine grounded in our knowledge base, a deterministic solar sizing calculator, a knowledge base viewer, a QA/test case reference view, and the safety and regulatory guardrails that sit behind every response. Things like lead capture, user accounts, payments, and installer dispatch are deliberately left out for now — see Section 9 for the full list of what we're not building yet.

2. Overall Description

2.1 Product Perspective

Solari is a new, self-contained web application — we didn't build it on top of an existing product. The frontend is served as static assets and talks to the backend through exactly two REST endpoints (`/api/chat` and `/api/recommend`), nothing more. On the backend side, we integrate with Google's Gemini API for the conversational reasoning and native tool calling, and lean on local Markdown/PDF files as the actual source of truth for anything domain-specific.

2.2 User Classes and Characteristics

User Class	Characteristics
Prospective solar customer	A homeowner or business owner exploring whether solar is viable for their property, with little to no technical background in electrical engineering.
Returning / evaluating user	A user comparing system types (on-grid, hybrid, off-grid) or panel/inverter options before making a purchase decision.
Project evaluator / administrator	A team member or reviewer who inspects the knowledge base, the recorded test cases, and overall chatbot behavior for correctness.

2.3 Operating Environment

- Client: any modern desktop or mobile web browser with JavaScript enabled.
- Server: Python 3.10+ runtime hosting a FastAPI application via Uvicorn.
- External dependency: Google Gemini API (model: gemini-3.1-flash-lite) reachable over HTTPS.

3. Functional Requirements

We've grouped these by module so it's easier to trace a requirement back to the part of the app it belongs to. Priority follows the usual MoSCoW scale — Must, Should, Could — based on how central each item is to Solari actually working as intended.

3.1 Conversational Chat Interface

ID	Requirement	Priority
FR-1	The system shall provide a persistent chat thread where the user can type a message and submit it by clicking Send or pressing Enter.	Must
FR-2	The system shall display a live character counter (0/500) and enforce a 500-character input limit.	Must
FR-3	The system shall show a typing indicator while waiting for the assistant's reply.	Should
FR-4	The system shall render the assistant's reply as formatted Markdown (headings, bold text, bullet points, tables).	Must
FR-5	The system shall offer clickable suggestion chips (e.g. “Size my system”, “Compare panels”) that populate and send a pre-written question.	Should
FR-6	The system shall display a network-error banner if the chat request fails, without losing the user's typed message history.	Must

3.2 AI Consultation Engine

ID	Requirement	Priority
FR-7	The system shall answer user questions using only facts contained in the local knowledge base (hardware, systems, and operations research documents).	Must
FR-8	The system shall decline to answer questions unrelated to solar energy, power electronics, or utility billing, with a clear scope statement.	Must
FR-9	The system shall explain the difference between Net Billing and Net Metering under the NEPRA 2026 framework whenever billing/ROI questions arise.	Must
FR-10	The system shall never state or imply that a solar system will reduce a customer's electricity bill to zero.	Must
FR-11	The system shall detect emergency indicators (sparking, burning smell, smoke, fault codes) in a user message and respond with the mandatory shutdown/safety protocol.	Must
FR-12	The system shall warn against DIY high-voltage wiring and instruct users to contact a certified installer for hands-on electrical work.	Must

3.3 Deterministic Sizing Calculator

ID	Requirement	Priority
FR-13	The system shall expose a calculator modal where a user enters average monthly usage (kWh) and optional backup hours.	Must
FR-14	The system shall compute recommended system size (kW), panel count, required roof area (sq ft), and battery capacity (kWh) from fixed engineering constants.	Must
FR-15	The system shall enforce a minimum recommended system size of 3.0 kW regardless of low usage input.	Must
FR-16	When the AI engine detects a sizing request in chat, it shall invoke the same calculation function as a tool call rather than estimating the numbers itself.	Must
FR-17	The system shall append the mandatory engineering-estimate disclaimer to every sizing or cost-related response.	Must
FR-18	The system shall reject invalid inputs (zero or negative usage, negative backup hours) with a clear error message.	Must

3.4 Knowledge Base and Test Case Access

ID	Requirement	Priority
FR-19	The system shall provide a Knowledge Base view listing the source PDF documents the assistant is grounded on, each downloadable/viewable in a new tab.	Should
FR-20	The system shall provide a Test Cases view linking to the QA spreadsheet used to validate assistant responses.	Could

3.5 General Interface Features

ID	Requirement	Priority
FR-21	The system shall allow the user to toggle between light and dark themes.	Could
FR-22	The system shall sanitize all rendered chat content to prevent HTML/script injection from either user input or model output.	Must

4. Non-Functional Requirements

Attribute	Requirement
Performance	A typical chat response (excluding external Gemini API latency) should be returned within a few seconds under normal load; the deterministic /api/recommend endpoint should respond near-instantly.
Usability	The interface must be usable by non-technical homeowners: plain language, a visible input limit, and clear error banners instead of raw error codes.
Reliability	If the Gemini API key is missing or the AI call fails, the backend shall return a graceful, descriptive message instead of crashing the request.

Attribute	Requirement
Security	User input rendered in the chat UI must be sanitized (e.g. via DOMPurify) before insertion into the DOM to prevent script injection.
Regulatory compliance	All billing, ROI, and interconnection guidance must reflect the NEPRA 2026 Net-Billing framework and must never promise a zero electricity bill.
Maintainability	Domain knowledge must be editable by updating the Markdown/PDF files in the knowledge base without changing application code.
Portability	The backend must run on any platform supporting Python 3.10+ (Windows, macOS, Linux) with no OS-specific dependencies.

5. External Interface Requirements

5.1 User Interface

It's a single-page app: a left navigation panel (Chat, Knowledge base, Test cases, Leads), a main chat panel with the message composer, and a modal dialog that pops up for the sizing calculator. Nothing more complicated than that by design — we wanted it to feel approachable to someone who's never talked to a chatbot about their electricity bill before.

5.2 Application Programming Interfaces

Endpoint	Purpose
GET /health	Returns service status; used for uptime checks.
POST /api/recommend	Accepts monthly_units and backup_hours; returns a deterministic sizing recommendation with no AI call.
POST /api/chat	Accepts a user message and preferred system_type; returns the AI-generated reply, invoking the sizing tool automatically when needed.

5.3 External Service Interface

The backend talks to the Google Gemini API (google-genai SDK, model gemini-3.1-flash-lite) over HTTPS, using native function/tool calling to trigger the sizing calculation whenever it's needed — the model requests it, our backend runs the real math, and the result gets handed back for the model to explain in plain language.

6. System / Technology Requirements

Layer	Technology
Frontend	HTML5, CSS3, vanilla JavaScript; marked.js for Markdown rendering; DOMPurify for sanitization.
Backend	Python 3.10+, FastAPI, Uvicorn ASGI server, python-dotenv for configuration.
AI provider	Google Gemini API via the google-genai SDK (model: gemini-3.1-flash-lite).
Knowledge base	Local Markdown files (hardware, systems, operations research) plus reference PDFs.
Testing artifact	A prepared Excel workbook of 30 test cases validating knowledge-base grounded responses.

7. Constraints

- The assistant has to answer strictly from the knowledge base we gave it — no inventing specs, prices, or policies, even if it sounds plausible.
- A valid Gemini API key needs to be sitting in the backend environment, or the conversational endpoint simply won't work.
- Sizing calculations always run through our calculate_solar_recommendation function — we deliberately never let the language model do this math on its own, since we can't fully trust it to stay consistent.
- Every sizing or cost response has to carry the mandatory disclaimer text, word for word — this isn't optional.

8. Assumptions and Dependencies

- We're assuming the user at least knows their monthly electricity bill or unit consumption — without that, a sizing estimate isn't really possible.
- We're treating our three knowledge-base documents as the up-to-date, authoritative source for hardware, systems, and operations facts — if they go stale, the assistant's answers will too.
- Both sides need internet access: the user to reach the frontend, and our server to reach the Gemini API.

9. Out of Scope

- User authentication, accounts, or saved conversation history — not part of this version.
- Payment processing, invoicing, or any kind of checkout flow.
- Automated installer dispatch or scheduling — the “Leads” nav item is just a placeholder for now, not a working feature.
- A physical site survey, shading analysis, or structural assessment — we always point users to a certified installer for that, per the mandatory disclaimer.